Hands-on Praktik Pertemuan 6

Implementasi Blockchain Sederhana dengan Python

Teknologi Blockchain - Program Studi Informatika - Institut Teknologi Tangerang Selatan

Komponen	Keterangan
Topik RPKPS	Implementasi Blockchain Sederhana
Sub-CPMK	SUB-IF219-2-3 - menerapkan mekanisme pembentukan blok dan validasi rantai pada program seder
Metode	Guided coding, tugas terstruktur, self-directed learning, diskusi reflektif
Durasi	100-150 menit, dapat disesuaikan dengan alokasi perkuliahan
Tools	Python 3.10+ atau Google Colab; editor bebas; terminal/command prompt

Dokumen ini adalah panduan praktik lengkap. Slide presentasi hanya memberikan preview alur, cuplikan kode, dan checkpoint kelas.

1. Tujuan Praktikum

Setelah menyelesaikan hands-on ini, mahasiswa diharapkan mampu:

- Merepresentasikan block sebagai struktur data yang memiliki index, timestamp, transactions, previous_hash, nonce, dan hash.
- Mengimplementasikan fungsi hash SHA-256 untuk membuat sidik jari digital block.
- Membuat genesis block dan menambahkan block baru ke chain.
- Mensimulasikan mining sederhana dengan mencari nonce yang menghasilkan hash sesuai difficulty.
- Memvalidasi chain dengan mengecek integritas hash dan hubungan previous_hash.
- Membuktikan efek tampering melalui eksperimen perubahan transaksi pada block lama.

Hubungan dengan materi sebelumnya

Pertemuan ini menggabungkan konsep dari pertemuan sebelumnya: ledger dan transaksi, block dan hash, digital signature secara konseptual, serta konsensus/mining. Fokus praktik hari ini adalah mengubah konsep tersebut menjadi prototype kecil yang bisa dijalankan.

Catatan dosen: tekankan bahwa prototype ini bukan blockchain produksi. Tujuannya adalah memahami mekanisme dasar secara operasional.

2. Skenario Praktik

Mahasiswa akan membuat blockchain kecil untuk mencatat transaksi sederhana, misalnya transfer nilai antarpengguna. Program akan membuat block baru dari pending transactions, menambang block dengan difficulty rendah, menambahkan block ke chain, dan memvalidasi chain.

Objek	Peran dalam program
Transaction	Data sederhana berisi pengirim, penerima, dan amount.
Block	Wadah transaksi dan metadata yang menghasilkan hash.
Chain	List block yang saling terhubung melalui previous_hash.
Nonce	Angka percobaan yang diubah saat mining.
Difficulty	Target jumlah leading zero pada hash block.
Validator	Fungsi yang mengecek apakah setiap block dan link antarblock masih valid.

3. Persiapan Lingkungan

Pilihan A - Google Colab

- Buka Google Colab dan buat notebook baru.
- Buat beberapa cell sesuai langkah pada dokumen ini.
- Jalankan setiap cell dari atas ke bawah.
- Screenshot output sebelum dan sesudah tampering.

Pilihan B - Local Python

- Pastikan Python 3.10 atau lebih baru sudah terpasang.
- Buat folder week6-simple-blockchain.
- Buat file simple_blockchain.py.
- Jalankan perintah: python simple_blockchain.py.

```
# Cek versi Python
python --version

# Membuat folder praktik
mkdir week6-simple-blockchain
cd week6-simple-blockchain

# Membuat file program
# Windows: notepad simple_blockchain.py
# Mac/Linux: nano simple_blockchain.py atau gunakan VS Code
```

4. Step 1 - Import dan Helper Waktu

Program membutuhkan hashlib untuk SHA-256, json untuk representasi deterministic, dataclass untuk struktur Block, dan datetime untuk timestamp.

```
from dataclasses import dataclass, field
from datetime import datetime
from hashlib import sha256
import json


def now():
    return datetime.utcnow().isoformat(timespec="seconds") + "Z"
```

Gunakan UTC agar timestamp tidak bergantung pada zona waktu komputer masing-masing mahasiswa.

5. Step 2 - Membuat Class Block

Class Block menyimpan data utama yang diperlukan untuk membuat rantai. Field hash akan diisi setelah block dihitung atau ditambang.

```
@dataclass
class Block:
    index: int
    timestamp: str
    transactions: list
    previous_hash: str
    nonce: int = 0
    hash: str = ""

    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
        }, sort_keys=True)
        return sha256(block_string.encode()).hexdigest()
```

Field	Fungsi
index	Nomor urut block di dalam chain.
timestamp	Waktu block dibuat.
transactions	Daftar transaksi yang dicatat pada block.
previous_hash	Hash dari block sebelumnya, digunakan untuk mengunci keterkaitan chain.
nonce	Angka yang dicoba berulang kali saat mining.
hash	Sidik jari digital block setelah dihitung.

6. Step 3 - Membuat Class SimpleBlockchain

Class SimpleBlockchain mengatur chain, pending transactions, difficulty, genesis block, mining, dan validasi.

```
class SimpleBlockchain:
    def __init__(self, difficulty=3):
        self.chain = [self.create_genesis_block()]
        self.pending_transactions = []
        self.difficulty = difficulty

    def create_genesis_block(self):
        block = Block(
            index=0,
            timestamp=now(),
            transactions=[{"system": "Genesis Block"}],
            previous_hash="0"
        )
        block.hash = block.calculate_hash()
        return block
```

Genesis block adalah block pertama. Karena tidak memiliki block sebelumnya, previous_hash biasanya diisi 0 atau konstanta khusus.

7. Step 4 - Menambahkan Transaksi

Transaksi tidak langsung masuk ke chain. Transaksi dikumpulkan lebih dulu di pending_transactions, kemudian dimasukkan ke block berikutnya saat mining.

```
    def add_transaction(self, sender, receiver, amount):
        tx = {
            "from": sender,
            "to": receiver,
            "amount": amount
        }
        self.pending_transactions.append(tx)
```

Checkpoint

Tambahkan minimal dua transaksi sebelum melakukan mining. Contoh: Alice ke Bob 10, Bob ke Carol 3.

8. Step 5 - Mining Block

Mining pada praktik ini dilakukan dengan mencari nonce hingga hash block diawali sejumlah nol sesuai difficulty.

```
    def mine_block(self, block):
        target = "0" * self.difficulty
        block.hash = block.calculate_hash()

        while not block.hash.startswith(target):
            block.nonce += 1
            block.hash = block.calculate_hash()

        print(f"Block {block.index} mined")
        print(f"  nonce = {block.nonce}")
        print(f"  hash  = {block.hash}")
```

Gunakan difficulty=2 atau difficulty=3 di kelas agar proses cepat. Difficulty yang terlalu besar dapat membuat praktik menjadi lambat.

9. Step 6 - Menambang Pending Transactions

Fungsi berikut membuat block baru dari pending_transactions, menghubungkannya ke block terakhir, menambangnya, lalu menambahkannya ke chain.

```
    def mine_pending_transactions(self):
        if not self.pending_transactions:
            print("No pending transactions to mine.")
            return

        block = Block(
            index=len(self.chain),
            timestamp=now(),
            transactions=self.pending_transactions,
            previous_hash=self.chain[-1].hash
        )
        self.mine_block(block)
        self.chain.append(block)
        self.pending_transactions = []
```

Diskusi singkat

Mengapa transaksi dikumpulkan dulu, bukan langsung dimasukkan satu per satu ke block? Hubungkan dengan konsep mempool dan efisiensi pembuatan block.

10. Step 7 - Validasi Chain

Validasi minimal memeriksa dua hal: hash block masih benar dan previous_hash masih menunjuk ke hash block sebelumnya.

```
    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]

            if current.hash != current.calculate_hash():
                print(f"Invalid hash at block {current.index}")
                return False

            if current.previous_hash != previous.hash:
                print(f"Broken link at block {current.index}")
                return False

            if not current.hash.startswith("0" * self.difficulty):
                print(f"Proof-of-work invalid at block {current.index}")
                return False

        return True
```

Validasi genesis block dapat dibuat lebih ketat pada versi lanjutan, tetapi untuk praktik ini fokus pada block setelah genesis.

11. Step 8 - Menampilkan Isi Chain

Fungsi print_chain membantu mahasiswa membaca isi chain dan melihat hubungan previous_hash.

```
    def print_chain(self):
        for block in self.chain:
            print("-" * 72)
            print(f"Block Index     : {block.index}")
            print(f"Timestamp       : {block.timestamp}")
            print(f"Previous Hash   : {block.previous_hash}")
            print(f"Nonce           : {block.nonce}")
            print(f"Hash            : {block.hash}")
            print(f"Transactions    : {block.transactions}")
```

12. Step 9 - Main Program

Tambahkan blok kode berikut di bagian bawah file untuk menjalankan eksperimen penuh.

```
if __name__ == "__main__":
    blockchain = SimpleBlockchain(difficulty=3)

    blockchain.add_transaction("Alice", "Bob", 10)
    blockchain.add_transaction("Bob", "Carol", 3)
    blockchain.mine_pending_transactions()

    blockchain.add_transaction("Carol", "Dave", 2)
    blockchain.add_transaction("Dave", "Eve", 1)
    blockchain.mine_pending_transactions()

    print("
Initial blockchain:")
    blockchain.print_chain()
    print("
Initial chain valid?", blockchain.is_chain_valid())

    print("
Tampering block 1...")
    blockchain.chain[1].transactions[0]["amount"] = 999

    print("After tamper, chain valid?", blockchain.is_chain_valid())
```

13. Expected Output

Hash yang muncul pada komputer mahasiswa dapat berbeda karena timestamp dan nonce berbeda. Namun pola output harus mirip dengan berikut.

```
Block 1 mined
  nonce = 814
  hash  = 000a6a0f2c...<dipotong>
Block 2 mined
  nonce = 2101
  hash  = 00099b1e84...<dipotong>

Initial blockchain:
------------------------------------------------------------------------
Block Index     : 0
Timestamp       : 2026-09-27T01:20:01Z
Previous Hash   : 0
Nonce           : 0
Hash            : 7e7b2...<dipotong>
Transactions    : [{'system': 'Genesis Block'}]
...

Initial chain valid? True

Tampering block 1...
Invalid hash at block 1
After tamper, chain valid? False
```

14. Eksperimen Wajib

Eksperimen	Instruksi	Evidence
E1 - Hash berubah	Ubah amount transaksi pada block 1 dari 10 menjadi 999.	Screenshot validasi False.
E2 - Mining ulang satu block	Setelah tamper, hitung ulang hash block 1 saja. Periksa apakah block 2 masih valid.	Catat apa yang berhasil dan apa yang masih gagal.
E3 - Difficulty	Jalankan program dengan difficulty 2 dan 4. Bandingkan nonce dan waktu mining.	Tabel perbandingan difficulty, nonce, dan obs
E4 - Tambah transaksi	Tambahkan 3 transaksi baru dan mine block 3.	Screenshot chain yang berisi minimal 4 block

15. Pertanyaan Analisis

Jawab pertanyaan berikut pada reflection.md:

- Mengapa calculate_hash() harus menggunakan data block yang stabil dan deterministic?
- Apa peran previous_hash dalam menjaga keterkaitan antarblock?
- Mengapa setelah satu transaksi diubah, chain menjadi tidak valid?
- Apa perbedaan mining dan validation pada simulasi ini?
- Mengapa prototype ini belum dapat disebut blockchain produksi?

16. Simulasi Tambahan: Tiga Node

Bagian ini opsional bila waktu kelas mencukupi. Tujuannya memperlihatkan bahwa satu node yang di-tamper dapat berbeda dari node lain yang masih memiliki chain valid.

```
import copy

node_a = blockchain
node_b = copy.deepcopy(blockchain)
node_c = copy.deepcopy(blockchain)

# Node B mencoba memalsukan transaksi
node_b.chain[1].transactions[0]["amount"] = 5000

print("Node A valid?", node_a.is_chain_valid())
print("Node B valid?", node_b.is_chain_valid())
print("Node C valid?", node_c.is_chain_valid())
```

Diskusi: pada blockchain nyata, node yang memiliki chain tidak valid akan ditolak oleh node lain. Namun keputusan chain utama tetap memerlukan aturan consensus dan network propagation.

17. Troubleshooting

Masalah	Kemungkinan penyebab	Solusi
Program sangat lama	Difficulty terlalu besar.	Gunakan difficulty 2 atau 3.
Hash selalu berubah	Timestamp ikut berubah saat calculate_hash dipanggil ulang.	Pastikan timestamp disimpan sebagai field tetap di block.
Tamper tidak terdeteksi	Validasi tidak menghitung ulang hash.	Tambahkan check current.hash != current.calculate_hash()
Broken link tidak muncul	previous_hash tidak dibandingkan dengan previous.hash.	Tambahkan check current.previous_hash != previous.hash.
Output berbeda antar mahasiswa	Timestamp dan nonce berbeda.	Normal, yang penting hash memenuhi target dan validasi s

18. Format Pengumpulan

Kumpulkan folder atau repository dengan struktur berikut:

```
week6-simple-blockchain/
|-- simple_blockchain.py
|-- README.md
|-- reflection.md
|-- screenshots/
|   |-- before_tamper.png
|   |-- after_tamper.png
|   |-- difficulty_comparison.png
```

File	Isi minimal
simple_blockchain.py	Kode final yang dapat dijalankan.
README.md	Cara menjalankan, versi Python, dan ringkasan fitur.
reflection.md	Jawaban pertanyaan analisis dan ringkasan eksperimen.
screenshots	Bukti eksekusi sebelum tamper, sesudah tamper, dan eksperimen difficulty.

19. Rubrik Formatif

Kriteria	Bobot	Indikator
Struktur implementasi	30%	Block, chain, genesis block, hash, previous_hash, transaksi, dan mining tersedia s
Validasi chain	25%	Program mampu mendeteksi chain valid dan tidak valid setelah perubahan data.
Kualitas kode	20%	Kode modular, readable, dan dapat dijalankan tanpa error.
Pengujian	15%	Tersedia skenario uji, screenshot, dan interpretasi hasil.
Dokumentasi	10%	README dan reflection menjelaskan desain dan hasil eksperimen.

20. Full Source Code

```
from dataclasses import dataclass
from datetime import datetime
from hashlib import sha256
import json


def now():
    return datetime.utcnow().isoformat(timespec="seconds") + "Z"


@dataclass
class Block:
    index: int
    timestamp: str
    transactions: list
    previous_hash: str
    nonce: int = 0
    hash: str = ""

    def calculate_hash(self):
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "transactions": self.transactions,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
        }, sort_keys=True)
        return sha256(block_string.encode()).hexdigest()


class SimpleBlockchain:
    def __init__(self, difficulty=3):
        self.chain = [self.create_genesis_block()]
        self.pending_transactions = []
        self.difficulty = difficulty

    def create_genesis_block(self):
        block = Block(0, now(), [{"system": "Genesis Block"}], "0")
        block.hash = block.calculate_hash()
        return block

    def add_transaction(self, sender, receiver, amount):
        tx = {"from": sender, "to": receiver, "amount": amount}
        self.pending_transactions.append(tx)

    def mine_block(self, block):
        target = "0" * self.difficulty
        block.hash = block.calculate_hash()
        while not block.hash.startswith(target):
            block.nonce += 1
            block.hash = block.calculate_hash()
        print(f"Block {block.index} mined")
        print(f"  nonce = {block.nonce}")
        print(f"  hash  = {block.hash}")

    def mine_pending_transactions(self):
        if not self.pending_transactions:
            print("No pending transactions to mine.")
            return
        block = Block(
            index=len(self.chain),
            timestamp=now(),
            transactions=self.pending_transactions,
            previous_hash=self.chain[-1].hash
        )
        self.mine_block(block)
        self.chain.append(block)
        self.pending_transactions = []

    def is_chain_valid(self):
        for i in range(1, len(self.chain)):
            current = self.chain[i]
            previous = self.chain[i - 1]
            if current.hash != current.calculate_hash():
                print(f"Invalid hash at block {current.index}")
                return False
            if current.previous_hash != previous.hash:
                print(f"Broken link at block {current.index}")
```

```
                return False
            if not current.hash.startswith("0" * self.difficulty):
                print(f"Proof-of-work invalid at block {current.index}")
                return False
        return True

    def print_chain(self):
        for block in self.chain:
            print("-" * 72)
            print(f"Block Index     :

{block.index}")
            print(f"Timestamp       : {block.timestamp}")
            print(f"Previous Hash   : {block.previous_hash}")
            print(f"Nonce           : {block.nonce}")
            print(f"Hash            : {block.hash}")
            print(f"Transactions    : {block.transactions}")


if __name__ == "__main__":
    blockchain = SimpleBlockchain(difficulty=3)
    blockchain.add_transaction("Alice", "Bob", 10)
    blockchain.add_transaction("Bob", "Carol", 3)
    blockchain.mine_pending_transactions()
    blockchain.add_transaction("Carol", "Dave", 2)
    blockchain.add_transaction("Dave", "Eve", 1)
    blockchain.mine_pending_transactions()
    print("\nInitial blockchain:")
    blockchain.print_chain()
    print("\nInitial chain valid?", blockchain.is_chain_valid())
    print("\nTampering block 1...")
    blockchain.chain[1].transactions[0]["amount"] = 999
    print("After tamper, chain valid?", blockchain.is_chain_valid())
```

21. Exit Ticket

Tuliskan jawaban singkat sebelum keluar kelas:

- Apa fungsi previous_hash dalam blockchain?
- Mengapa perubahan satu transaksi dapat membuat chain tidak valid?
- Apa perbedaan mining dan validation dalam simulasi hari ini?

Selesai.